Vector Index Tuning

recall, latency and memory: pick two, tune the third

Watch the 4-min video

In one line: HNSW's **M** and **ef_construction** fix graph quality at build time and **ef_search** trades recall for latency per query; memory is usually the real limit, so **quantise** (int8, binary, PQ) and **re-score** the top candidates with full vectors; and use engines that **filter during the graph search**.

1 From demo to 50 million vectors

Semantic search works in the demo. Then you load 50 million 1,000-dimension embeddings: the index needs over 200 GB of RAM, queries take 200 ms, and some obvious matches go missing. Vector indexes don't tune themselves. A handful of parameters decide **recall**, **latency** and **cost**, and the only reliable way to set them is to measure on your own data.

2 HNSW's three knobs



Changing M or ef_construction means rebuilding the index; ef_search can change per request.

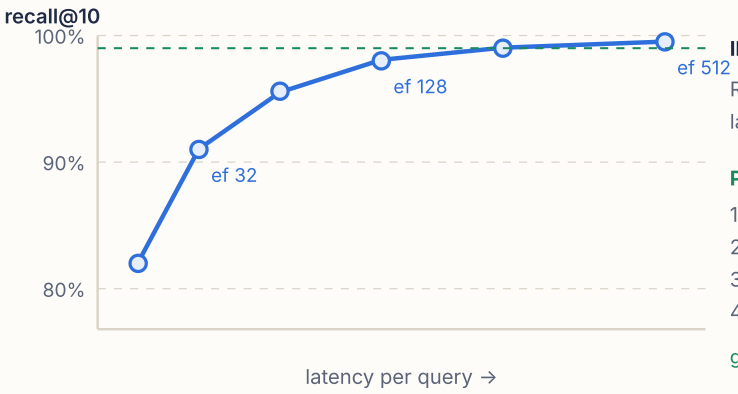
Figure 1. Build-time knobs shape the graph; the runtime knob decides how much of it each query explores.

- **M** is how many neighbours each node links to (layer 0 usually allows 2·M). More links give a better-connected graph and higher recall, especially for high-dimensional or clustered data, but more memory and slower builds and inserts.
- **ef_construction** is the candidate-list size used while choosing each new node's links. Higher values find better neighbours and a higher recall ceiling, paid once at build time. Worth raising if builds aren't on a critical path.
- **ef_search** (called `num_candidates` per shard in Elasticsearch kNN queries, `hnsw.ef_search` in pgvector) is the candidate-list size during a query. It must be at least k. It is the knob you tune first, because it needs no rebuild.

Engine	M default	ef_construction default	query-time knob (default)
hnswlib	16	200	`ef` (10)
Lucene / Elasticsearch	16 (`m`)	100	`num_candidates` per query
pgvector	16 (`m`)	64	`hnsw.ef_search` (40)

Quote these as starting points only; in an interview it is stronger to say *'I'd benchmark M and ef on our data against exact search'* than to quote magic numbers.

3 Trading recall for latency



Illustrative shape, not a benchmark.
Recall rises fast, then flattens;
latency keeps growing roughly linearly.

Procedure:

1. sample ~1,000 real queries
2. exact top-10 by brute force
3. sweep ef_search, measure recall, p99
4. pick the lowest ef that hits target

green line = recall target (e.g. 95%)

Figure 2. The recall–latency curve for ef_search. Choose the knee that meets your recall target, not the maximum.

Recall is always measured against **exact search on a sample**: run real queries brute force to get the true top 10, then compare. Watch **p99** latency, not just the mean, and repeat after big data changes. Sharding also matters: each shard returns its own top candidates, so more shards mean more total work per query but smaller graphs; searching fewer, larger segments is faster than many small ones (in Lucene each segment has its own graph, so force-merging read-only indexes helps).

4 Memory: the real limit

```
RAM ≈ N × dims × bytes per dim + N × ~2M × 4 B (links)
```

A 1,000-dim float32 vector is 4 KB, so 50 M of them are 200 GB before the graph. HNSW is fast only when its working set is in RAM (random access hops), so memory sets the hardware bill. Ways to shrink it:

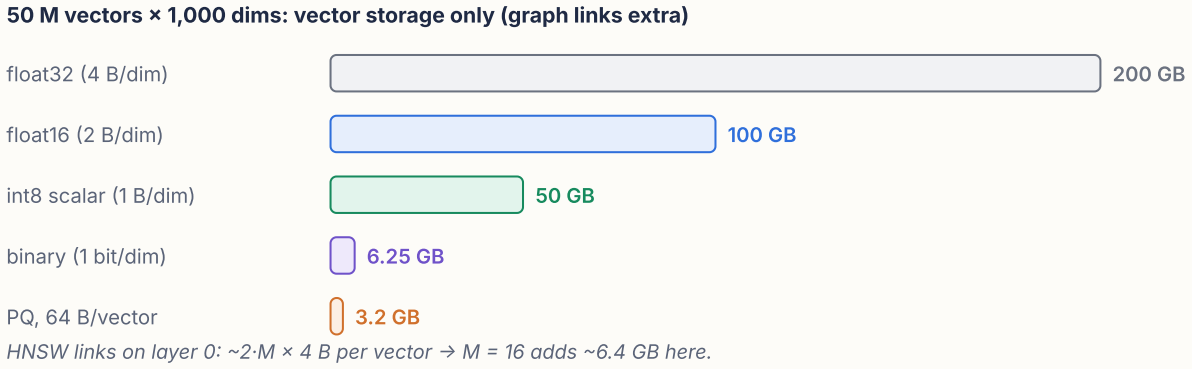


Figure 3. Same 50 M vectors under different encodings. Compression ratios are exact; the recall cost depends on the data and the model.

- **Scalar quantisation (int8)**: map each float dimension to 8 bits using per-dimension or per-segment min/max. 4× smaller, usually a small recall loss. Elasticsearch's `int8_hnsw` index option and OpenSearch/Faiss support it.
- **Binary quantisation**: 1 bit per dimension, 32× smaller, distances via fast bit operations; needs re-scoring and works best with high-dimensional models.
- **Product quantisation (PQ)**: split the vector into m sub-vectors (e.g. 64 chunks of ~16 dims), learn 256 centroids per chunk, store one byte per chunk: 64 bytes instead of 4,000, about 60×. Lossier; usually combined with IVF (IVF-PQ in Faiss).

- **Fewer dimensions**: some models (e.g. OpenAI's text-embedding-3) let you request shorter vectors; a smaller model may be enough.
- **Re-scoring**: search the compressed index for, say, 100 candidates, then recompute exact distances with the full float vectors kept on disk and return the best 10. This recovers most of the lost recall for a little extra I/O.

5 Filtered vector search

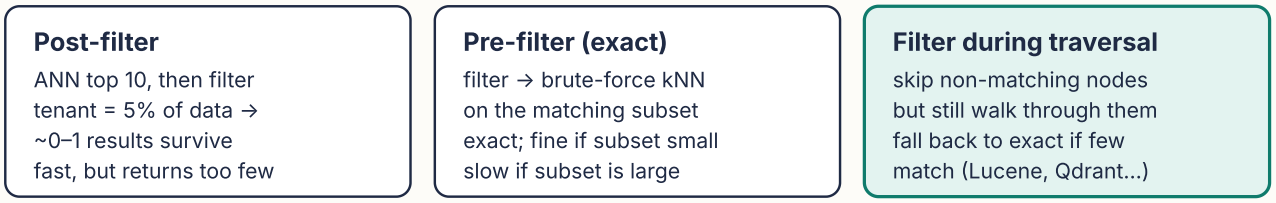


Figure 4. Three ways to combine 'similar to X' with 'tenant = 42 and language = en'.

Real queries have filters: this tenant, this language, in stock. **Post-filtering** the ANN result can leave nothing when the filter is selective; asking for more candidates only partly helps. **Pre-filtering** then brute-forcing is exact and cheap only if the subset is small. Modern engines filter **during** the graph walk: non-matching nodes are traversed but not returned, and when the filter matches only a few documents, the engine switches to exact search over them (Lucene does this based on the filter's cardinality). For multi-tenant systems with very selective tenant filters, a separate index or partition per large tenant (lesson 41) is often the cleanest answer.

WORKED EXAMPLE: 4× LESS MEMORY WITH INT8

A team stores 40 M product embeddings at 1,000 dims: 40 M × 4 KB = **160 GB** of floats, plus ~5 GB of links at M = 16. They switch to int8 scalar quantisation and keep the float vectors on disk: in-memory vectors drop to **40 GB**. Measured recall@10 dips slightly against exact search. So each query fetches the top **100** candidates from the int8 index and re-scores them with the full vectors from disk (100 × 4 KB = 400 KB of reads, mostly from page cache), returning the best 10. Recall@10 is back to **98%**, p99 latency barely moves, and the cluster runs on roughly a quarter of the RAM.

TRADE-OFFS

- **Higher M**: better recall on hard data, but more memory per vector and slower inserts.
- **Higher ef_search**: more recall, linearly more latency and CPU.
- **Quantisation**: 4–60× less memory, some recall loss; re-scoring buys most of it back.
- **In-RAM HNSW vs disk-based indexes** (DiskANN-style): latency vs cost at billion scale.

COMMON MISTAKES

- Tuning without a ground-truth recall measurement against exact search.
- Raising ef_search to fix bad results that are really caused by the embedding model or chunking.
- Post-filtering a selective tenant filter and returning empty pages.
- Sizing RAM for the vectors but forgetting graph links, replicas and headroom for merges/rebuilds.

6 Key terms

M: max links per node in the HNSW graph

ef_construction: candidate list size while building

ef_search: candidate list size per query (≥ k)

Product quantisation: sub-vectors coded by learned centroids

Scalar quantisation: float → int8 per dimension, 4× smaller

Re-scoring: exact distances on top candidates

7 Interview questions

Q1. How do you trade recall for latency in a vector index? BASIC

M and ef_construction shape the graph at build time; ef_search sets how much each query explores. Measure recall against exact search and pick the cheapest setting that meets the target.

Q2. How do you cut vector memory? BASIC

Quantise: int8 scalar for about 4×, binary or product quantisation for much more, and re-score the top candidates with full vectors.

Q3. Recall is 88% and the target is 95%. What do you try, in order? SCENARIO

Raise ef_search (no rebuild) and check the latency; if it plateaus below target, rebuild with higher M or ef_construction; also check quantisation and re-score more candidates.

Q4. Estimate RAM for 100 M × 768-dim float vectors with HNSW M = 16. SCENARIO

Vectors: 100 M × 768 × 4 B ≈ 307 GB; links ≈ 100 M × 32 × 4 B ≈ 13 GB; so ~320 GB per copy before replicas and headroom, or ~90 GB with int8.

Q5. Tenant-filtered queries return 2 results instead of 10. Why? SCENARIO

Post-filtering: the ANN top candidates mostly belong to other tenants. Filter during traversal, raise candidates, or use per-tenant indexes/partitions for large tenants.

Q6. Why can a higher ef_search not fix poor results? TRICKY

It only brings ANN closer to exact search; if exact search itself returns poor matches, the problem is the embeddings, chunking or query, not the index.

Q7. Why does quantisation plus re-scoring work so well? TRICKY

Quantised distances are good enough to get the true neighbours into a top-100 shortlist; exact distances then fix their order, so you pay full-precision cost only for 100 vectors.

Next: SD-P-011 · Hybrid Search backendroadmap.com